System Overview

Introduction

The Liquid Level Sensor System is a mechatronics training platform designed to demonstrate the integration of sensors, controls, and user calibration in a process environment. Its primary function is to detect the liquid level in a container of water and provide visual feedback through onboard indicators.

Hardware Architecture

The system is built around the **Adafruit Circuit Playground Express (CPX)**, which serves as the central controller. The CPX utilizes its capacitive touch sensor at pin **A1**, connected via a wired alligator clip secured to the rim of the container, to detect changes in capacitance caused by the presence of liquid. This sensing method allows the system to identify when the liquid reaches a defined level.d level.

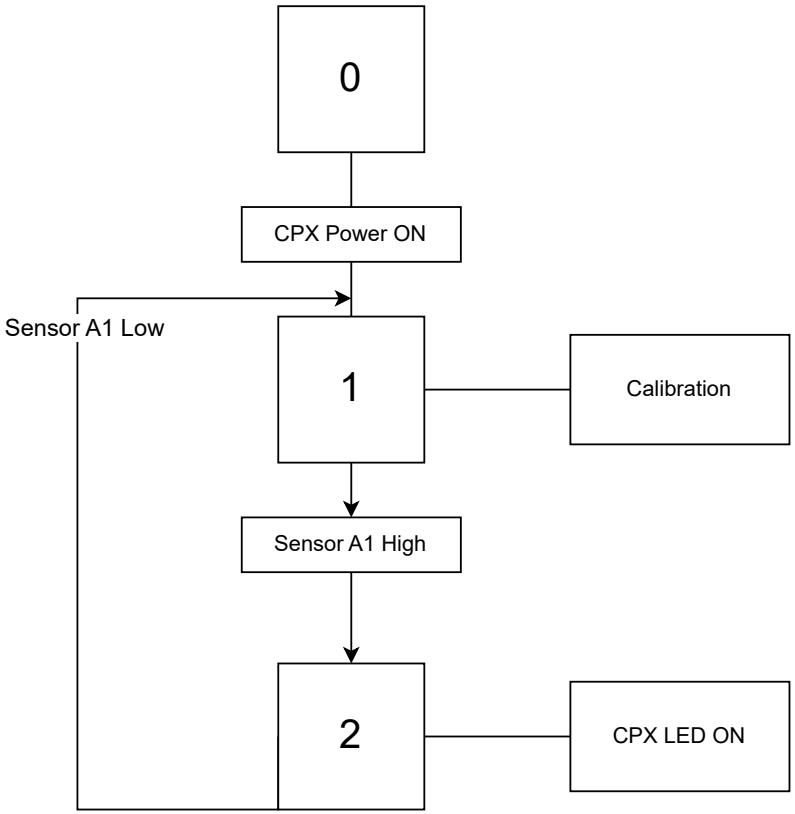
Power and Feedback

Power is supplied by a **3.7V battery**, ensuring portability and safe operation in laboratory settings. The CPX's **NeoPixel LEDs** provide real-time feedback and are also used during manual calibration. Calibration enables adjustment for environmental conditions (humidity, temperature, container material) and for different liquids with varying dielectric properties, ensuring reliable detection across multiple scenarios.enarios.

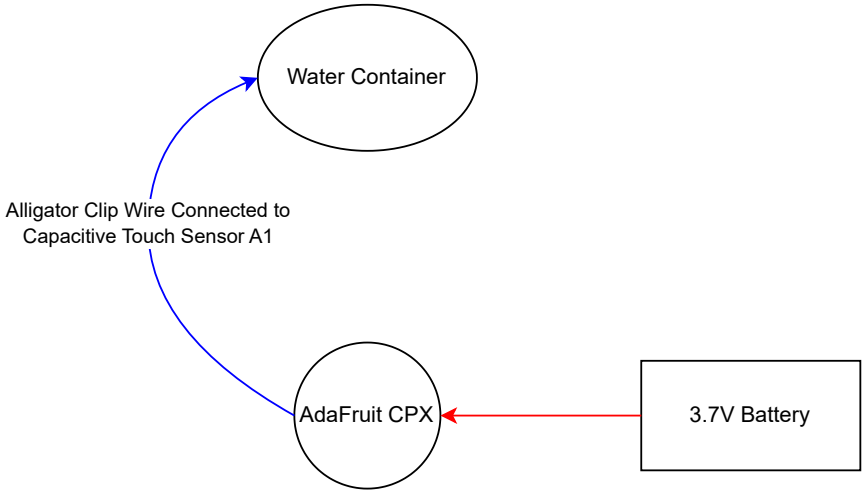
System Integration

This system highlights the integration of hardware (CPX, sensors, battery), software (calibration routines, LED control), and process control concepts. It serves as a practical demonstration of how mechatronic systems combine sensing, computation, and actuation to achieve robust functionality in real-world applications.

Sequential Function Chart



EMI Chart



Tag Table

STEP	Component	Address
1	Water Hose Flow	
2	Capacitive Touch Sensor	A1

Code

```
import time
import board
import touchio
from adafruit_circuitplayground import cp

# Initialize the capacitive touch sensor on A1
touch_pad = touchio.TouchIn(board.A1)

# Settings for visualization and tuning
PIXEL_COUNT = 10
BAR_COLOR = (255, 160, 0) # amber for bar
THRESHOLD_COLOR = (255, 255, 255) # white marker (not flashing)

# Initialize NeoPixels
cp.pixels.brightness = 0.3
cp.pixels.fill((0, 0, 0))

# --- Auto-calibration on startup ---
# Show amber while calibrating the dry baseline
cp.pixels[0] = (255, 140, 0) # amber
samples = []
for _ in range(40): # ~2 seconds at 0.05s/sample
samples.append(touch_pad.raw_value)
time.sleep(0.05)

# Use median for robustness to spikes
samples.sort()
dry_baseline = samples[len(samples) // 2]

# Load saved margin if present (plain text to avoid needing the json module)
default_margin = 1000
threshold_margin = default_margin
try:
```

```
try:
with open("cal.txt", "r") as f:
txt = f.read().strip()
threshold_margin = int(txt)
except Exception:
pass

# Threshold derived from baseline + margin
detection_threshold = dry_baseline + threshold_margin

# End of calibration indicator: turn off pixel 0 (no white flash)
cp.pixels[0] = (0, 0, 0)

# Debounce and UI state
wet_count = 0
dry_count = 0
is_wet = False

prev_btn_a = False
prev_btn_b = False
ab_hold_started_at = None
MARGIN_STEP_COUNTS = 100 # counts to change per button press
BAR_SPAN_COUNTS = 2000 # map 0..BAR_SPAN across the 10 pixels


def clamp_int(value, lo, hi):
"""Clamp integer value to [lo, hi]."""
if value < lo:
return lo
if value > hi:
return hi
return value


def draw_bar_graph(raw_value, baseline, margin):
"""Draw bar of raw_value above baseline and white threshold marker.

raw_value: current capacitive reading
baseline: dry baseline captured on boot
margin: threshold offset above baseline
```

"""
# Compute bar level relative to baseline
delta = raw_value - baseline
delta = clamp_int(delta, 0, BAR_SPAN_COUNTS)
level = int((delta * PIXEL_COUNT) / BAR_SPAN_COUNTS) # 0..10

# Pixel index for threshold marker from margin position
m = clamp_int(margin, 0, BAR_SPAN_COUNTS)
t_idx = int((m * PIXEL_COUNT) / BAR_SPAN_COUNTS)
if t_idx >= PIXEL_COUNT:
t_idx = PIXEL_COUNT - 1

# Draw the bar with a solid white threshold marker at t_idx
for i in range(PIXEL_COUNT):
if i == t_idx:
cp.pixels[i] = THRESHOLD_COLOR
elif i < level:
cp.pixels[i] = BAR_COLOR
else:
cp.pixels[i] = (0, 0, 0)

while True:
raw_value = touch_pad.raw_value

# Update detection with hysteresis
if raw_value > detection_threshold:
wet_count = min(wet_count + 1, 3)
dry_count = 0
else:
dry_count = min(dry_count + 1, 3)
wet_count = 0

new_is_wet = wet_count >= 2
if new_is_wet != is_wet:
is_wet = new_is_wet

# Show bar and threshold marker every loop
draw_bar_graph(raw_value, dry_baseline, threshold_margin)

```
# Red LED indicates detection state (works without using NeoPixel colors)
cp.red_led = is_wet

# Button edge detection for tuning
btn_a_pressed = cp.button_a
btn_b_pressed = cp.button_b

if btn_a_pressed and not prev_btn_a:
new_margin = threshold_margin - MARGIN_STEP_COUNTS
threshold_margin = clamp_int(new_margin, 0, 8000)
detection_threshold = dry_baseline + threshold_margin
# brief dim green blink on pixel 0
cp.pixels[0] = (0, 50, 0)
if btn_b_pressed and not prev_btn_b:
new_margin = threshold_margin + MARGIN_STEP_COUNTS
threshold_margin = clamp_int(new_margin, 0, 8000)
detection_threshold = dry_baseline + threshold_margin
# brief dim green blink on pixel 0
cp.pixels[0] = (0, 50, 0)

# Long press A+B to save margin (write plain integer to cal.txt)
if btn_a_pressed and btn_b_pressed:
if ab_hold_started_at is None:
ab_hold_started_at = time.monotonic()
elif time.monotonic() - ab_hold_started_at > 1.0:
try:
with open("cal.txt", "w") as f:
f.write(str(int(threshold_margin)))
except Exception:
pass
# Flash green across ring to confirm save
for _ in range(2):
for i in range(PIXEL_COUNT):
cp.pixels[i] = (0, 100, 0)
time.sleep(0.15)
for i in range(PIXEL_COUNT):
cp.pixels[i] = (0, 0, 0)
time.sleep(0.1)
ab_hold_started_at = None
```

```
ab_hold_started_at = None
else:
ab_hold_started_at = None

prev_btn_a = btn_a_pressed
prev_btn_b = btn_b_pressed

time.sleep(0.06)
```

Controller Overview

The Adafruit Circuit Playground Express (CPX) serves as the central controller for the Liquid Level Sensor System. It integrates sensing, computation, and user interface functions into a single compact platform, enabling reliable detection and calibration of liquid levels.

The CPX is powered by a 3.7V rechargeable battery, providing portable operation and safe voltage levels for laboratory use. Its capacitive touch sensor at pin A1 is configured as the primary input, connected via an alligator clip to the rim of the container. This sensor detects changes in capacitance caused by the presence of liquid, allowing the controller to determine when the water reaches the defined level.

The CPX also incorporates NeoPixel LEDs and onboard controls, which serve as both output indicators and calibration tools. The LEDs provide immediate visual feedback of sensor status, while the onboard buttons allow manual adjustment of sensitivity to account for environmental conditions (humidity, temperature, container material) and variations in liquid properties. This ensures accurate and adaptable operation across different scenarios.

Internally, the CPX executes a control program written in CircuitPython which manages sensor input, calibration routines, and output display. The program is modular, allowing easy modification for expanded functionality, such as multi-level detection or integration with external actuators.

By combining sensing, processing, and feedback in one device, the CPX demonstrates the principles of mechatronic control: integration of hardware, software, and user interaction to achieve robust system performance.